Servicio Nacional de Aprendizaje

SENA

Tecnólogo en Análisis y Desarrollo de Software

Evidencia GA11-220501098-AA1-EV02

Aplicación de buenas prácticas de calidad documentadas en las disciplinas de calidad de software

Alumno:

Gerson Jonnathan López Oviedo

Ficha 3186647

Instructor:

Arnaldo Alfonso Montiel Brun

Proyecto:

Trueque Match G.L. Software Solutions

Bogotá, Colombia 04/octubre/2026

Contenido

1. Introducción 3

2. Instrumento de Diseño de Calidad de Software 4

3. ¿Qué Significa Desarrollar Software? 5

4. Etapas del Desarrollo de Software 6

4.1. Análisis y Levantamiento de Requisitos 6

4.2. Diseño de la Arquitectura y la Interfaz 6

4.3. Programación y Codificación 6

4.4. Pruebas y Control de Calidad 7

4.5. Despliegue 7

4.6. Mantenimiento y Actualización 7

5. Modelos de Desarrollo de Software 8

5.1. Modelo en Cascada 8

5.2. Modelo Ágil 8

5.3. Modelo en Espiral 8

5.4. Modelo Aplicado en Trueque Match 8

6. Tipos de Desarrollo de Software 9

6.1. Desarrollo Móvil 9

6.2. Desarrollo Corporativo 9

6.3. Desarrollo de Software a Medida 9

6.4. Desarrollo de Software con Inteligencia Artificial 9

7. Calidad del Software 10

7.1. ¿Qué es la Calidad de Software? 10

7.2. QA Management, QA Assurance y QA Control 10

8. Estándares de Calidad del Software 11

9. Estándar Aplicado en el Proyecto Trueque Match 12

10. Buenas Prácticas de Calidad Aplicadas en Trueque Match 13

11. Conclusiones 15

12. Referencias 16

1. Introducción

Este documento reúne la evidencia GA11-220501098-AA1-EV02, correspondiente a la aplicación de buenas prácticas de calidad dentro de las disciplinas de calidad de software. El propósito es doble: por un lado, explicar de forma clara y sencilla los conceptos fundamentales del desarrollo y la calidad del software qué significa desarrollar software, cuáles son sus etapas, qué modelos y tipos de desarrollo existen, y qué estándares internacionales regulan la calidad; por otro lado, mostrar cómo esos conceptos se aplicaron de manera real en el proyecto Trueque Match, una aplicación web y móvil para el intercambio de productos, servicios, conocimientos y experiencias sin dinero, desarrollada como proyecto formativo del programa Tecnólogo en Análisis y Desarrollo de Software.

A lo largo del documento se parte siempre de la definición general de cada tema y luego se conecta con una situación concreta del proyecto Trueque Match, de manera que el lector pueda ver no solo la teoría, sino su aplicación práctica en un software real, construido con PHP, MySQL y probado mediante la herramienta Postman.

2. Instrumento de Diseño de Calidad de Software

Siguiendo el formato GFPI-F-135 V02 del SENA para el diseño de instrumentos de calidad, a continuación se presenta la tabla que se utilizó para evaluar de forma ordenada la calidad del software Trueque Match. El instrumento identifica el criterio de calidad evaluado, la técnica o herramienta empleada para evaluarlo y el resultado real obtenido durante el desarrollo del proyecto.

Criterio de calidad evaluado	Instrumento / técnica utilizada	Resultado obtenido
Funcionalidad (CRUD completo de cada módulo)	Colección de pruebas en Postman (GET, POST, PUT, DELETE) para Ofertas, Solicitudes, Trueques, Evaluaciones, Notificaciones, Reportes y Favoritos	10 de 10 endpoints probados y funcionando correctamente, incluyendo la API de edición de perfil (api_usuario.php) verificada en esta sesión; evidencia en video de pruebas Postman.
Seguridad (validación de entradas)	Revisión manual de código de los archivos api_*.php, comparando el método PUT contra el método POST de cada API	Se detectó que en api_ofertas.php el método POST no escapa el campo categoria antes de insertarlo en la consulta SQL; el PUT sí lo hace. Riesgo de inyección SQL identificado y pendiente de corrección
Consistencia de datos (nombres de tablas)	Comparación del comportamiento del motor de base de datos en Windows/XAMPP frente a Linux/InfinityFree	Se corrigieron 10 archivos PHP que referenciaban tablas en mayúscula, evitando fallas al migrar de un sistema operativo que no distingue mayúsculas a uno que sí lo hace

Criterio de calidad evaluado	Instrumento / técnica utilizada	Resultado obtenido
Usabilidad (datos mostrados al usuario)	Pruebas manuales de navegación con distintos usuarios registrados	Se corrigieron secciones del panel (Mi Perfil, Mis Trueques, Trueques Pendientes, avatar del menú) que mostraban información fija o de ejemplo en lugar de los datos reales del usuario conectado
Mantenibilidad (documentación)	Elaboración de README técnico, manual técnico y manual de usuario	Documentación publicada en el repositorio de evidencias, cubriendo prerrequisitos, arquitectura, diccionario de datos y guía de instalación

Este instrumento no es un ejercicio teórico: cada fila corresponde a una verificación que se hizo realmente sobre el código del repositorio del proyecto, incluyendo un hallazgo de seguridad que a la fecha de este documento sigue pendiente de corrección, tal como se detalla más adelante en la sección de buenas prácticas aplicadas.

3. ¿Qué Significa Desarrollar Software?

Desarrollar software significa construir un programa o conjunto de programas que resuelven un problema o una necesidad de una persona, una empresa o una comunidad, siguiendo un proceso ordenado: primero se entiende el problema, luego se diseña una solución, después se escribe el código que la hace funcionar, se comprueba que funcione bien y finalmente se pone a disposición de quien la va a usar. No es simplemente "escribir código": es un proceso que incluye pensar, planear, construir, probar y mantener un producto a lo largo del tiempo.

En el caso de Trueque Match, desarrollar el software significó partir de una necesidad real permitir que las personas intercambien productos, servicios, conocimientos y experiencias sin usar dinero y convertirla en una aplicación web y móvil funcional, con una base de datos que organiza

la información, un servidor que procesa las solicitudes (API) y una interfaz donde el usuario puede registrarse, publicar ofertas, solicitar trueques y calificar la experiencia.

4. Etapas del Desarrollo de Software

El desarrollo de cualquier software, sin importar su tamaño, suele pasar por seis grandes etapas. A continuación se explica cada una en términos simples y se describe cómo se vivió esa etapa en el proyecto Trueque Match.

4.1. Análisis y Levantamiento de Requisitos

Es la etapa donde se investiga y se define exactamente qué necesita el usuario, qué problema se quiere resolver y qué funciones debe tener el software. Aquí se responde a la pregunta "¿qué vamos a construir y para quién?".

Aplicación en Trueque Match: se definió que la aplicación debía permitir el registro de usuarios, la publicación de ofertas (productos, servicios, conocimientos o experiencias), la gestión de solicitudes de trueque, la evaluación entre usuarios y un sistema de notificaciones y reportes, todo pensado para el contexto colombiano de economía colaborativa.

4.2. Diseño de la Arquitectura y la Interfaz

Es la etapa donde se decide cómo va a estar organizado el software por dentro (qué partes tiene, cómo se comunican) y cómo se va a ver por fuera (los colores, la tipografía, la distribución de los botones y las pantallas).

Aplicación en Trueque Match: se definió una arquitectura basada en PHP y MySQL/MariaDB para el servidor, con diez tablas normalizadas en tercera forma normal (tipo_usuario, usuario, oferta, trueque, evaluacion, solicitud, notificacion, reporte, usuario_oferta, administrador) y una identidad visual propia: rojo #C0392B, fondo oscuro #111111, texto #F5F0EB y las fuentes Bebas Neue y Nunito, aplicada tanto en la versión web como en la app móvil construida con React Native y Expo.

4.3. Programación y Codificación

Es el momento en que, con base en el diseño ya definido, se escribe el código real que hace funcionar cada parte del sistema: formularios, botones, cálculos, conexión con la base de datos, etc.

Aplicación en Trueque Match: se construyeron las páginas de la app web (index, ofertas, dashboard, login, registro) y diez APIs en PHP (login, registro, ofertas, solicitudes, trueques, evaluaciones, favoritos, notificaciones, reportes y usuario) que permiten que la aplicación web y la app móvil lean y escriban datos en la base de datos mediante peticiones GET, POST, PUT y DELETE.

4.4. Pruebas y Control de Calidad

Es la etapa donde se revisa que todo lo construido funcione correctamente, se buscan errores (bugs) y se corrigen antes de que el software llegue al usuario final.

Aplicación en Trueque Match: cada una de las diez APIs se probó manualmente con Postman en sus operaciones de creación, lectura, actualización y eliminación, lo que permitió encontrar y corregir errores reales: tablas escritas en mayúscula que fallaban en Linux, páginas de login y registro que en realidad eran maquetas sin conectar, datos de ejemplo (María, Pedro, Laura, Juan) que aparecían fijos en el panel en lugar de los datos del usuario real, y un hallazgo de seguridad en la API de ofertas que aún está pendiente de corregir.

4.5. Despliegue

Es la etapa en la que el software deja de estar solo en la computadora del desarrollador y se publica en un servidor real, para que cualquier persona pueda usarlo desde internet.

Aplicación en Trueque Match: el proyecto se publicó en el hosting gratuito InfinityFree, subiendo manualmente los archivos (ya que un git push al repositorio de GitHub no actualiza automáticamente el hosting), y se generó un archivo APK real de la app móvil con seis pantallas funcionales mediante Expo.

4.6. Mantenimiento y Actualización

Es la etapa posterior al lanzamiento, en la que el software se sigue corrigiendo, mejorando y adaptando a nuevas necesidades o problemas que aparecen con el uso real.

Aplicación en Trueque Match: después de publicado el sitio, se han seguido corrigiendo errores reales encontrados en producción (como los datos de ejemplo hardcodeados), se documentó un plan de migración y respaldo de datos, y queda como tarea activa de mantenimiento corregir la

validación de seguridad pendiente en la API de ofertas y resolver la alerta de "sitio peligroso" que Google Safe Browsing muestra actualmente sobre el dominio del proyecto.

5. Modelos de Desarrollo de Software

Un modelo de desarrollo de software es la forma en que se organiza, en el tiempo, el trabajo de construir el software: en qué orden se hacen las etapas, si se repiten o no, y qué tan flexible es el plan frente a los cambios. A continuación se explican los tres modelos más usados.

5.1. Modelo en Cascada

Es el modelo más clásico: las etapas se hacen una después de la otra, en un orden fijo (análisis, diseño, programación, pruebas, despliegue, mantenimiento), y normalmente no se vuelve a una etapa anterior una vez terminada. Funciona bien cuando los requisitos están muy claros desde el principio y no van a cambiar.

5.2. Modelo Ágil

En lugar de hacer todo el software de una sola vez, se construye en pequeñas partes (llamadas iteraciones o sprints), entregando funciones que ya sirven cada pocas semanas, recibiendo retroalimentación y ajustando el rumbo sobre la marcha. Es el modelo más usado hoy en día porque se adapta bien a los cambios.

5.3. Modelo en Espiral

Combina ideas del modelo en cascada y del modelo ágil: el proyecto avanza en "vueltas" repetidas (como una espiral), y en cada vuelta se analiza el riesgo, se diseña, se construye y se prueba una parte del software, antes de pasar a la siguiente vuelta. Se usa mucho en proyectos grandes donde el riesgo de que algo salga mal es alto.

5.4. Modelo Aplicado en Trueque Match

El desarrollo de Trueque Match siguió principalmente un modelo ágil e iterativo: el proyecto avanzó módulo por módulo (primero ofertas, luego solicitudes, luego trueques, luego evaluaciones, etc.), cada módulo se probó con Postman apenas estuvo listo, y los errores encontrados en una sesión de pruebas se corrigieron y se volvieron a probar en la sesión siguiente,

en lugar de dejar todas las pruebas para el final del proyecto, como ocurriría en un modelo en cascada puro.

6. Tipos de Desarrollo de Software

Además del modelo de trabajo, el software se puede clasificar según el tipo de solución que se construye. A continuación se explican cuatro tipos comunes y cuál aplica a Trueque Match.

6.1. Desarrollo Móvil

Es la construcción de aplicaciones pensadas para celulares y tabletas, que se instalan como app (Android o iOS) o se generan con herramientas como React Native, que permiten escribir el código una sola vez y convertirlo en una app real para varios dispositivos. Trueque Match tiene su propia app móvil, construida con React Native y Expo, con un archivo APK real de seis pantallas, probada en dos celulares Xiaomi distintos (Redmi 9 y Redmi 10C).

6.2. Desarrollo Corporativo

Es el desarrollo de software pensado para resolver necesidades internas de una empresa u organización (por ejemplo, sistemas de nómina, inventarios o gestión de clientes), generalmente usado solo por los empleados de esa organización.

6.3. Desarrollo de Software a Medida

Es el desarrollo de una solución construida desde cero para un cliente o un problema específico, en lugar de usar un programa genérico ya hecho. Trueque Match es, en este sentido, un desarrollo a medida: no se adaptó un sistema de comercio electrónico ya existente, sino que se diseñó y programó un modelo de datos y unas reglas de negocio propias para el intercambio sin dinero.

6.4. Desarrollo de Software con Inteligencia Artificial

Es el uso de herramientas de inteligencia artificial como apoyo durante el proceso de construcción del software: para generar código, documentación, diagramas o para ayudar a encontrar y explicar errores. En Trueque Match, la inteligencia artificial se usó como herramienta de apoyo para generar diagramas técnicos (casos de uso, componentes, paquetes, mapa de navegación), redactar documentación técnica y acompañar la revisión de código en busca de

errores, mientras que las decisiones finales y la validación de lo construido siguieron siendo responsabilidad del desarrollador.

7. Calidad del Software

7.1. ¿Qué es la Calidad de Software?

La calidad de software es el grado en que un programa cumple con lo que el usuario necesita (que haga bien lo que tiene que hacer), funciona de manera confiable (que no falle o se caiga), es fácil de usar, es seguro (que no exponga información ni tenga puertas abiertas a ataques) y es fácil de mantener y mejorar en el futuro. En otras palabras, un software de calidad no es solo uno que "funciona", sino uno que funciona bien, de forma segura y sostenible en el tiempo.

7.2. QA Management, QA Assurance y QA Control

Dentro de la disciplina de calidad de software existen tres niveles de trabajo que se suelen confundir entre sí, pero que cumplen funciones distintas:

- QA Management (Gestión de la calidad): es la planeación general de cómo se va a garantizar la calidad en todo el proyecto qué normas se van a seguir, qué se va a revisar y con qué frecuencia. Es la parte de "organizar el plan".
- QA Assurance (Aseguramiento de la calidad): son las actividades preventivas que buscan evitar errores antes de que ocurran, como definir buenas prácticas de programación, usar nombres de tablas en minúscula de forma consistente, o documentar el proceso. Busca que el error no aparezca.
- QA Control (Control de calidad): son las actividades que revisan el producto ya construido para encontrar errores que sí ocurrieron, como las pruebas con Postman de cada API. Busca encontrar y corregir el error que ya está presente.

En Trueque Match, la gestión de calidad se reflejó en la decisión de probar cada API antes de darla por terminada; el aseguramiento de calidad se vio en prácticas como escapar los textos antes de insertarlos en la base de datos para evitar inyección SQL; y el control de calidad se evidenció directamente en los bugs reales que se encontraron y corregios mediante las pruebas en Postman, como los datos de ejemplo hardcodeados en el panel de usuario.

8. Estándares de Calidad del Software

Existen varios estándares y modelos internacionales, creados por organizaciones como ISO (International Organization for Standardization) e IEEE (Institute of Electrical and Electronics Engineers), que sirven de guía para que el software que se construye en cualquier parte del mundo cumpla con un nivel mínimo de calidad. A continuación se explican, en lenguaje sencillo, los más relevantes para este proyecto.

Estándar	Qué significa, en términos simples
ISO 9001	Es la norma internacional de gestión de la calidad más conocida en el mundo. No habla solo de software: sirve para cualquier tipo de organización. Dice, en palabras simples, que una empresa debe tener procesos claros, documentados y que se revisen constantemente para asegurar que lo que entrega cumple lo que el cliente pidió.
ISO 10005:2018	Es una norma que da una guía sobre cómo hacer un "plan de calidad": un documento donde se explica qué se va a hacer, con qué recursos y bajo qué criterios, para asegurar la calidad de un producto, proyecto o contrato específico.
ISO/IEC 25000 (SQuaRE)	Es una familia de normas dedicada específicamente al software. Define qué características debe tener un software de calidad (que funcione bien, que sea confiable, que sea fácil de usar, que rinda bien, que sea seguro, que se pueda mantener) y da pautas para medirlas y evaluarlas de forma objetiva.
ISO/IEC 33000	Es una familia de normas que sirve para evaluar qué tan maduros son los procesos de una organización que desarrolla software, es decir, qué tan ordenados, repetibles y controlados son sus procesos de trabajo, no el producto final.
CMMI (Capability Maturity Model Integration)	Es un modelo (no una norma ISO) que mide el nivel de madurez de los procesos de una organización de software, desde procesos caóticos y poco definidos (nivel más bajo) hasta procesos optimizados y basados en mejora continua (nivel más alto).

Estándar	Qué significa, en términos simples
ISO/IEC 12207	Es la norma que describe el ciclo de vida completo del software: qué procesos existen desde que se concibe la idea hasta que el software deja de usarse, incluyendo análisis de requisitos, diseño, construcción, pruebas, despliegue y mantenimiento. Es, en esencia, la base formal de las "etapas del desarrollo de software" explicadas en este documento.
IEEE 730	Es un estándar que define qué debe contener un plan de aseguramiento de calidad de software (SQAP): qué se va a revisar, con qué criterios y quién es responsable de cada verificación.
ISO/IEC 5055	Es una norma más reciente (2021) que define medidas automatizadas para evaluar la calidad del código fuente directamente, usando herramientas de análisis estático que revisan seguridad, confiabilidad, eficiencia de desempeño y facilidad de mantenimiento del código sin necesidad de ejecutarlo.

9. Estándar Aplicado en el Proyecto Trueque Match

De todos los estándares descritos, los dos que mejor describen lo que realmente se hizo en Trueque Match son ISO/IEC 12207 e ISO/IEC 25000 (SQuaRE), por las siguientes razones:

- ISO/IEC 12207 se usó como marco de referencia para organizar el proyecto en las etapas descritas en este documento: análisis de requisitos, diseño de arquitectura, programación, pruebas, despliegue y mantenimiento, en ese orden y de forma iterativa.
- ISO/IEC 25000 (SQuaRE) se usó como referencia para decidir qué características de calidad evaluar del software ya construido: funcionalidad (CRUD completo probado en Postman), seguridad (revisión del escape de datos en las consultas SQL, donde se encontró una falla real aún sin corregir en el método POST de api_ofertas.php), usabilidad (que el panel mostrara los datos reales del usuario y no datos de ejemplo) y mantenibilidad (documentación técnica y manual de usuario).

Es importante ser honesto en este punto: Trueque Match es un proyecto formativo desarrollado por un único estudiante, por lo que no se aplicaron estos estándares de forma certificada ni con toda la rigurosidad que exigiría una empresa grande (por ejemplo, no se usó una herramienta automatizada de análisis de código bajo ISO/IEC 5055, ni se hizo una evaluación formal de madurez de procesos bajo ISO/IEC 33000). Lo que sí se hizo fue adoptar sus principios de manera práctica: documentar el ciclo de vida del proyecto, definir criterios de calidad claros y evaluarlos de forma honesta, dejando registrados tanto los aciertos como las fallas pendientes de corregir.

10. Buenas Prácticas de Calidad Aplicadas en Trueque Match

Más allá de la teoría, estas son las buenas prácticas de calidad que se aplicaron de forma concreta y verificable durante el desarrollo del proyecto:

- Control de versiones con Git y GitHub: cada cambio en el código quedó registrado en commits con mensajes descriptivos, lo que permite rastrear cuándo y por qué se modificó cada parte del sistema.
- Pruebas funcionales sistemáticas con Postman: las diez APIs del proyecto se probaron en sus operaciones GET, POST, PUT y DELETE antes de considerarlas terminadas, con una colección de pruebas exportable y un video de evidencia.
- Corrección iterativa basada en hallazgos reales: errores como tablas en mayúscula, páginas de login/registro sin conectar y datos de ejemplo hardcodeados se identificaron mediante pruebas reales y se corrigieron en commits posteriores, en lugar de darlos por solucionados sin verificarlos.
- Separación de responsabilidades en la validación de datos: se evidenció, comparando el método PUT (que sí valida la categoría contra una lista de valores permitidos) con el método POST (que no lo hace), que la falta de consistencia en la validación es en sí misma un riesgo de calidad, no solo de seguridad.
- Documentación técnica como práctica de mantenibilidad: se elaboraron un README técnico, un manual técnico y un manual de usuario, de forma que el conocimiento del proyecto no dependa únicamente de la memoria del desarrollador.

- Transparencia sobre pendientes: en lugar de presentar el proyecto como libre de errores, se mantiene un registro honesto de los hallazgos sin corregir (como la validación faltante en api_ofertas.php) como parte del proceso de mejora continua.

11. Conclusiones

La calidad de software no es un paso aislado al final del proyecto, sino una disciplina que atraviesa todas las etapas del desarrollo: se piensa desde el análisis de requisitos, se construye durante el diseño y la programación, se verifica en las pruebas y se sostiene en el mantenimiento. Los estándares internacionales como ISO/IEC 12207 e ISO/IEC 25000 no son solo documentos teóricos: ofrecen un lenguaje común y unos criterios concretos que, aplicados incluso de forma parcial y a escala de un proyecto formativo como Trueque Match, permiten evaluar de manera objetiva qué tan bien construido está un software y qué le falta por mejorar.

El desarrollo de Trueque Match mostró en la práctica que aplicar buenas prácticas de calidad no significa que el software quede perfecto, sino que los errores se encuentran, se documentan y se corrigen de forma ordenada, y que los pendientes como el hallazgo de seguridad sin corregir en la API de ofertas se reconocen abiertamente en lugar de ocultarse, lo cual es, en sí mismo, una buena práctica de calidad y de ética profesional.

12. Referencias

International Organization for Standardization. (2023). ISO/IEC 25000:2023 Systems and software engineering Systems and software Quality Requirements and Evaluation (SQuaRE). **https://www.iso.org/standard/91474.html**

International Organization for Standardization. (2018). ISO 10005:2018 Quality management systems Guidelines for quality plans. **https://www.iso.org/standard/70398.html**

Institute of Electrical and Electronics Engineers. (2014). IEEE 730-2014 IEEE Standard for Software Quality Assurance Processes. https://standards.ieee.org/ieee/730/970

Consortium for IT Software Quality. (2021). ISO/IEC 5055 Automated Source Code Quality Measures. **https://www.it-cisq.org/webinars/iso-5055-automated-source-code-quality-measures**

López Oviedo, G. J. (2026). Repositorio proyecto_sena [Repositorio de código]. GitHub. **https://github.com/lopezoviedogersonjonnathan-rgb/proyecto_sena**

López Oviedo, G. J. (2026). Repositorio evidencias-sena-trueque-match [Repositorio de evidencias]. **GitHub. https://github.com/lopezoviedogersonjonnathan-rgb/evidencias-sena-trueque-match**